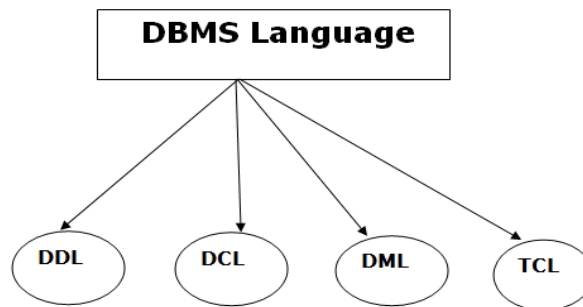


WEEK—5 Lecture-3 hr
DATABASE LANGUAGES

Types Of Database Languages:

- A DBMS has appropriate languages and interfaces to express database queries and updates.
- Database languages can be used to read, store and update the data in the database.

Types of Database Language



1. Data Definition Language

- **DDL** stands for **Data Definition Language**. It is used to define database structure or pattern.
- It is used to create schema, tables, indexes, constraints, etc. in the database.
- Using the DDL statements, you can create the skeleton of the database.
- Data definition language is used to store the information of metadata like the number of tables and schemas, their names, indexes, columns in each table, constraints, etc.

Here are some tasks that come under DDL:

- **Create:** It is used to create objects in the database.
- **Alter:** It is used to alter the structure of the database.
- **Drop:** It is used to delete objects from the database.
- **Truncate:** It is used to remove all records from a table.
- **Rename:** It is used to rename an object.
- **Comment:** It is used to comment on the data dictionary.

These commands are used to update the database schema that's why they come under Data definition language.

2. Data Manipulation Language

DML stands for **Data Manipulation Language**. It is used for accessing and manipulating data in a database. It handles user requests.

Here are some tasks that come under DML:

- **Select:** It is used to retrieve data from a database.
- **Insert:** It is used to insert data into a table.
- **Update:** It is used to update existing data within a table.
- **Delete:** It is used to delete all records from a table.

3. Data Control Language

- **DCL** stands for **Data Control Language**. It is used to retrieve the stored or saved data.
- The DCL execution is transactional. It also has rollback parameters.

(But in Oracle database, the execution of data control language does not have the feature of rolling back.)

Here are some tasks that come under DCL:

- **Grant:** It is used to give user access privileges to a database.
- **Revoke:** It is used to take back permissions from the user.

4. Transaction Control Language

TCL is used to run the changes made by the DML statement. TCL can be grouped into a logical transaction.

Here are some tasks that come under TCL:

- **Commit:** It is used to save the transaction on the database.
- **Rollback:** It is used to restore the database to original since the last Commit.

Integrity Constraints

Integrity constraints ensure that when the authorized users **modify** the database they do not disturb the **data consistency**. Integrity constraints are introduced while designing the **database schema**. The constraints are specified within the SQL DDL command like 'create table' and 'alter table' command.

The database system checks the specified constraint each time the database is updated. Any updations that cause modification to the database but, **violate** an integrity constraint,

are **rejected**. It is the **authorization and integrity manager** who test the integrity constraint each time a database is updated.

Types of Integrity Constraints in DBMS

- 1 Entity Integrity Constraints
- 2 Referential Integrity Constraints
- 3 Domain Constraints
- 4 Assertion

1. Entity Integrity Constraints

Entity integrity constraint ensures that the **primary key attribute** in a relation, should **not** accept a **null value**. This is because the primary key attribute value **uniquely** defines an entity in a relation. So, it being null would not work.

Whenever we declare any attribute in relation as the primary key attribute then it not necessary to specify it **explicitly** to be not null. In addition to primary key constraint, entity integrity constraint includes **unique key** constraint.

create table r (A1 D1, 2 D2,An Dn, unique (Ak1 , Ak2, . . . , Akm));

The set of attributes listed as unique forms the candidate key. No two entities (tuples) in a relation must have equal values for candidate key attribute. Candidate key attributes can accept 'null' values unless they are specifically declared to be 'not null'.

2. Referential Integrity Constraints

Referential integrity ensures that the **values** for a **set of attributes** in **one relation** must also appear the **same** for the **particular set attributes** in **another relation**.

Example:

```
create table Student
(Student_id varchar (5) ,
name varchar (20) not null,
depart_name varchar (20),
primary key (Student_id),
foreign key(depart_name) references Department);
```

Fig:1.1

The student relation has an attribute that refers to an attribute of the Department relation. So, the attribute or set of attributes in a relation referring to another relation in the database is called foreign key.

When the referential integrity constraint for a database is violated then the action causing the violation of referential integrity is rejected. It means if you insert a student tuple in the student relation with the depart_name value that doesn't exist in the depart_name attribute of some tuple in Department relation. Then this action would be rejected.

3. Domain Constraints

Domain constraint ensures that the value associated with an attribute is justifying its domain. Whenever we declare any relation to the database while declaring its attribute we specify a particular domain for each attribute.

The domain of an attribute specifies all the possible values that attribute can hold

The **name** attribute of student relation can only accept **string value of variable length**; it cannot accept an integer value or a date or time value.

There are three constraints which we can study under domain constraint.

1. Not null constraint	Null Values are allowed only if it is acceptable. But, there are some attributes which need not be null. We restrict the domain of that attribute for not accepting the null values. Create table Student(.....name varchar (20) not null....);
2. Default constraint	Using default value constraint, you are able to set a default value for an attribute. In case if you don't specify any value for an attribute on which default constraint is specified it holds the specified default value. Create table EMPLOYEE(..... Salary number(20) DEFAULT '0'....);
3. Check clause constraint	Check clause constraint ensures that when a new tuple is inserted in relation it must satisfy the predicate specified in the check clause. Create table Student(..... check (depart_name in('CS', 'EC', 'ME', 'CE')));

4. ASSERTION

An **assertion** is a **predicate** that contains a **condition** that must always be satisfied by the database. Before any modification to the database, the assertion is validated and if it does not satisfy the constraint the modification is rejected.

MySQL

MySQL OVERVIEW:

MySQL is an **open-source relational database management system (RDBMS)**. It is the most popular database system used with PHP. MySQL is developed, distributed, and supported by Oracle Corporation. The data in a MySQL database are stored in tables which consist of columns and rows.

MySQL Features:

The following are the most important features of MySQL:

- Relational Database Management System (RDBMS) MySQL is a relational database management system.
- Easy to use. MySQL is easy to use.
- It is secure.
- Client/ Server Architecture.

- Free to download.
- It is scalable.
- Speed.
- High Flexibility.
- Compatible on many operating systems.

MySQL Datatypes:

MySQL uses many different data types broken into three categories –

- Numeric
- Date and Time
- String Types.

Numeric Data Types

MySQL uses all the standard ANSI SQL numeric data types. The following list shows the common numeric data types.

- INT
- TINYINT
- SMALLINT
- MEDIUMINT
- BIGINT
- FLOAT(M,D)
- DOUBLE(M,D)
- DECIMAL(M,D)

Date and Time Types

The MySQL date and time datatypes are as follows –

- **DATE:** YYYY-MM-DD format.
- **DATETIME:** YYYY-MM-DD HH:MM:SS format.
- **TIMESTAMP:** YYYYMMDDHHMMSS format.
- **TIME:** HH:MM:SS format.
- **YEAR(M):** Stores a year in a 2-digit or a 4-digit format.

String Types

Although the numeric and date types are fun, most data you'll store will be in a string format. This list describes the common string datatypes in MySQL.

- **CHAR(M)** – A fixed-length string between 1 and 255 characters in length.
- **VARCHAR(M)** – A variable-length string between 1 and 255 characters in length.
- **BLOB or TEXT** – A field with a maximum length of 65535 characters. BLOBs are "Binary Large Objects" and are used to store large amounts of binary data, such as images or other types of files. Fields defined as TEXT also hold large amounts of data. The difference between the two is that the sorts and comparisons on the stored data are case sensitive on BLOBs and are not case sensitive in TEXT fields. You do not specify a length with BLOB or TEXT.
- **TINYBLOB or TINYTEXT** – A BLOB or TEXT column with a maximum length of 255 characters.
- **MEDIUMBLOB or MEDIUMTEXT** – A BLOB or TEXT column with a maximum length of 16777215 characters.
- **LOB or LONGTEXT** – A BLOB or TEXT column with a maximum length of 4294967295 characters.
- **ENUM** – An enumeration, which is a fancy term for list. When defining an ENUM, you are creating a list of items from which the value must be selected (or it can be NULL). For example, if you wanted your field to contain "A" or "B" or "C", you would define your ENUM as ENUM ('A', 'B', 'C') and only those values (or NULL) could ever populate that field.

MySQL Standardization guidelines:

- Do not ever give anyone (except MySQL root accounts) access to the user table in the mysql system database.
- Learn how the MySQL access privilege system works.
- Do not store cleartext passwords in your database.

PostgreSQL

PostgreSQL OVERVIEW:

PostgreSQL (pronounced as **post-gress-Q-L**) is an open source relational database management system (DBMS) developed by a worldwide team of volunteers. PostgreSQL is not controlled by any corporation or other private entity and the source code is available free of charge.

PostgreSQL Features:

- Free to download.
- Compatible on several operation systems.
- Compatible with various programming languages.
- Compatible with Data Integrity.
- Support multiple features of SQL.
- Compatible with multiple data types.
- Highly extensible.
- Secure.
- Highly Reliable.

PostgreSQL Datatypes:

PostgreSQL supports a wide set of Data Types. Besides, users can create their own custom data type using *CREATE TYPE* SQL command. There are different categories of data types in PostgreSQL. They are discussed below.

Numeric Types:

Numeric types consist of two-byte, four-byte, and eight-byte integers, four-byte and eight-byte floating-point numbers, and selectable-precision decimals.

Monetary Types:

The *money* type stores a currency amount with a fixed fractional precision. Values of the *numeric*, *int*, and *bigint* data types can be cast to *money*. Using Floating point numbers is not recommended to handle money due to the potential for rounding errors.

Character Types:

character varying(n), varchar(n):variable-length with limit

character(n), char(n):fixed-length, blank padded

text:variable unlimited length

Binary Data Types

Bytea: allows storage of variable-length binary string.

Date/Time Types

PostgreSQL supports a full set of SQL date and time types. Dates are counted according to the Gregorian calendar.

Boolean Type

PostgreSQL provides the standard SQL type Boolean. The Boolean data type can have the states *true*, *false*, and a third state, *unknown*, which is represented by the SQL null value.

Enumerated Type

Enumerated (enum) types are data types that comprise a static, ordered set of values. They are equivalent to the enum types supported in a number of programming languages.

Unlike other types, Enumerated Types need to be created using CREATE TYPE command. This type is used to store a static, ordered set of values. For example compass directions, i.e., NORTH, SOUTH, EAST, and WEST or days of the week as shown below –

```
CREATE TYPE week AS ENUM ('Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun');
```

Enumerated, once created, can be used like any other types.

Geometric Type

Geometric data types represent two-dimensional spatial objects. The most fundamental type, the point, forms the basis for all of the other types.

Network Address Type

PostgreSQL offers data types to store IPv4, IPv6, and MAC addresses. It is better to use these types instead of plain text types to store network addresses.

Bit String Type

Bit String Types are used to store bit masks. They are either 0 or 1. There are two SQL bit types: **bit(n)** and **bit varying(n)**, where n is a positive integer.

Text Search Type

This type supports full text search, which is the activity of searching through a collection of natural-language documents to locate those that best match a query.

XML Type

The XML data type can be used to store XML data.

JSON Type

The *json* data type can be used to store JSON (JavaScript Object Notation) data.

Array Type

PostgreSQL gives the opportunity to define a column of a table as a variable length multidimensional array. Arrays of any built-in or user-defined base type, enum type, or composite type can be created.

Composite Types

This type represents a list of field names and their data types, i.e., structure of a row or record of a table.

Range Types

Range types represent data types that uses a range of data.

Object Identifier Types

Pseudo Types

UUID Type